

SIMATS ENGINEERING

ASSESSMENT TOOL 2 - Technical Presentation

COURSE NAME: Data Structure

COURSE CODE: CSA0316

Assessment Tool Weightage: 30%

COURSE OUTCOME COVERED

CO1: *Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity. (BL3, BL4)*

Questions: EACH QUESTIONS CARRIES: 10 MARKS

Duration : ONE HOUR

S.No	List of Questions
19	A compiler checks whether parentheses in a program are balanced during syntax analysis. Identify the suitable data structure and justify based on operational requirements.

Code of Conduct:

I M.Vineela(192472071) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

A Compiler Checks Whether Parentheses In A Program Are Balanced During Syntax Analysis. Identify The

CSA0316- DATA STRUCTURES

Guided by:
Dr.Sunitharam C

Presented by:
M.Vineela(192472071)



Introduction to Syntax Analysis

- Syntax analysis is a phase in compiling that checks the structure of the of the source code.
- It ensures that the code adheres to the grammatical rules of the programming language.
- During this phase, various syntax-related checks, including parenthesis balancing, are performed.



Importance of Parentheses in Programming

- Parentheses are used to group expressions and define the order of operations.
- Correct placement of parentheses is crucial for the program's correctness and behavior.
- Misplaced or unbalanced parentheses can lead to syntax errors and program failures.



What Are Balanced Parentheses?

- Balanced parentheses mean that every opening parenthesis has a corresponding closing parenthesis.
- They are properly nested, ensuring the structure of expressions is valid.
- Examples include " $((a + b)$



Significance of Balancing During Syntax Analysis

- Balancing parentheses during syntax analysis helps catch syntax errors early.
- It prevents the parser from proceeding with invalid code, saving compilation time.
- Detecting unbalanced parentheses allows for meaningful error messages to be provided to programmers.



Techniques for Checking Parentheses

- The common method uses a stack data structure to verify balance.
- When an opening parenthesis is encountered, it is pushed onto the stack.
- When a closing parenthesis appears, the top of the stack is checked for a matching opening.



Algorithm for Parenthesis Balancing

- Initialize an empty stack at the start of the analysis.
- Scan the program character by character, pushing opening parentheses onto the stack.
- Pop from the stack when a closing parenthesis is found; if the stack is empty or mismatched, report an error.



Handling Errors in Parentheses

- If a closing parenthesis is found with an empty stack, it indicates an unbalanced parenthesis.
- If the stack is not empty after processing the entire program, it indicates missing closing parentheses.
- Error messages should specify the location and nature of the imbalance to aid debugging.



Implementation in Syntax Analyzers

- Most parsers use recursive descent or shift-reduce methods that incorporate parentheses checks.
- These methods integrate stack operations to validate parentheses during parsing.
- Balancing checks are often embedded as part of the grammar rules for expressions.



Benefits of Parentheses Checking

- Ensures syntactically correct code before further compilation stages.
- Helps prevent cascading errors that are harder to debug later.
- Provides immediate feedback to programmers about syntax issues, improving code quality.



Conclusion

- Checking for balanced parentheses is a vital part of syntax analysis in compilers.
- Using data structures like stacks makes this process efficient and reliable.
- Ensuring parentheses are balanced enhances the correctness and robustness of the compilation process.

Thank
You

The image features the words "Thank You" written in a fluid, cursive script. The text is centered and surrounded by several stylized, leaf-like or petal-like shapes, some of which are grouped together to form a larger, starburst-like pattern. The entire design is rendered in black lines on a light gray background.